

베이지안 기계학습

NumPyro (Pyro) 의 설치 및 사용법

조성일

인하대학교
통계학과

목차

1. NumPyro (Pyro) 의 소개
2. 설치 요구 사항
3. 설치 방법
4. 기본 사용법
5. 추가 예제

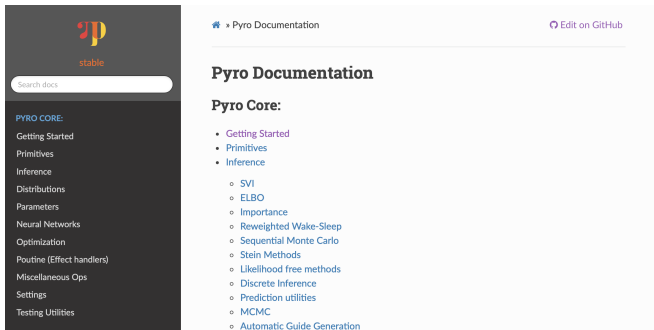
NumPyro (Pyro) 의 소개

NumPyro (Pyro) 란?

- ▶ NumPyro (Pyro) 는 Python을 위한 확률적 프로그래밍 라이브러리
- ▶ NumPyro는 JAX를 기반으로 빠른 자동 미분 및 벡터화 연산 제공
- ▶ 베이지안 모델링 및 추론에 적합

Pyro 홈페이지

▶ <https://docs.pyro.ai/en/stable/>



The screenshot displays the Pyro Documentation website. On the left is a dark sidebar with the Pyro logo (a stylized 'P' in orange and red) and the word 'stable' in orange. Below the logo is a search bar labeled 'Search docs'. The sidebar lists the following categories under 'PYRO CORE': Getting Started, Primitives, Inference, Distributions, Parameters, Neural Networks, Optimization, Routine (Effect handlers), Miscellaneous Ops, Settings, and Testing Utilities. The main content area has a header with 'Pyro Documentation' and a link to 'Edit on GitHub'. Below the header, the title 'Pyro Documentation' is repeated. Under the heading 'Pyro Core:', there is a list of links: Getting Started, Primitives, and Inference. The 'Inference' link is expanded, showing a sub-list of topics: SVI, ELBO, Importance, Reweighted Wake-Sleep, Sequential Monte Carlo, Stein Methods, Likelihood free methods, Discrete Inference, Prediction utilities, MCMC, and Automatic Guide Generation.

Pyro Documentation

Edit on GitHub

Pyro Documentation

Pyro Core:

- Getting Started
- Primitives
- Inference
 - SVI
 - ELBO
 - Importance
 - Reweighted Wake-Sleep
 - Sequential Monte Carlo
 - Stein Methods
 - Likelihood free methods
 - Discrete Inference
 - Prediction utilities
 - MCMC
 - Automatic Guide Generation

NumPyro 홈페이지

▶ <https://num.pyro.ai/en/latest/index.html#>



latest

Getting Started with NumPyro

API AND DEVELOPER REFERENCE

- Pyro Primitives
- Distributions
- Inference
- Effect Handlers
- Contributed Code
- Change Log

INTRODUCTORY TUTORIALS

- Bayesian Regression Using NumPyro
- Bayesian Hierarchical Linear Regression
- Example: Baseball Batting Average
- Example: Variational Autoencoder
- Example: Neal's Funnel
- Example: Stochastic Volatility
- Example: ProDLDA with Flax and Haiku
- Variationally Inferred Parameterization
- Automatic rendering of NumPyro models
- Bad posterior geometry and how to deal with it
- Truncated and folded distributions

NumPyro documentation

Edit on GitHub

NumPyro documentation

- Getting Started with NumPyro

API and Developer Reference

- Pyro Primitives
- Distributions
- Inference
- Effect Handlers
- Contributed Code
- Change Log



Bayesian Regression Using NumPyro



Bayesian Hierarchical Linear Regression

Baseball



Example: Baseball Batting Average



Example: Variational Autoencoder



Example: Neal's Funnel



Example: Stochastic Volatility



Example: ProDLDA with Flax and Haiku



Variationally Inferred Parameterization

NumPyro (Pyro) 의 주요 기능

- ▶ 다양한 확률 분포 및 샘플링 알고리즘 지원
- ▶ MCMC, 변분 추론 등 다양한 추론 방법 제공
- ▶ 사용자 친화적이며 유연한 API

NumPyro의 장점

- ▶ GPU 및 TPU 가속 지원으로 빠른 연산 가능
- ▶ 대규모 데이터셋 및 복잡한 모델에 적합
- ▶ Pyro 생태계와 호환 가능

참고: JAX 소개

JAX란?

- ▶ Python을 위한 고성능 수치 계산 라이브러리
- ▶ 자동 미분과 XLA 컴파일러 기반의 빠른 연산 제공
- ▶ 대규모 머신 러닝 모델과 과학적 연산에 최적화

주요 특징

- ▶ 자동 미분을 통한 효율적인 그래디언트 계산
- ▶ CPU, GPU, TPU 지원을 통한 빠른 연산 성능
- ▶ NumPy와 유사한 사용자 친화적 인터페이스
- ▶ 벡터화 및 병렬 처리 기능

설치 요구 사항

설치 요구 사항

Windows 운영 체제 사양

- ▶ Windows 10 이상 권장
- ▶ 충분한 하드 드라이브 공간
- ▶ 최소 4GB RAM 권장

Python 버전 요구 사항

- ▶ Python 3.6 이상
- ▶ 가상 환경 사용 권장 (예: conda, venv)

필요한 추가 소프트웨어 및 라이브러리

- ▶ NumPy
- ▶ SciPy
- ▶ JAX (NumPyro의 백엔드)
- ▶ 필요한 경우 CUDA (GPU 지원을 위해)

NumPyro 설치 방법

Python 설치 방법

- ▶ 공식 웹사이트(<https://www.python.org>)에서 Python을 다운로드하고 설치
- ▶ 설치 중 "Add Python to PATH" 옵션 선택

필요한 도구 설치 방법

- ▶ Jupyter Notebook: 커맨드 라인에서 'pip install notebook' 실행
- ▶ 필요한 경우, Anaconda 배포판 설치

Pip을 사용한 NumPyro (Pyro) 설치 명령어

Pyro 설치

- ▶ 커맨드 라인에서 'pip install pyro-ppl' 실행

NumPyro 설치

- ▶ 커맨드 라인에서 'pip install numpyro' 실행
- ▶ JAX와 함께 자동으로 설치됨

설치 확인 방법

- ▶ Python 셸 또는 Jupyter Notebook에서 'import pyro' 실행
- ▶ Python 셸 또는 Jupyter Notebook에서 'import numpyro' 실행
- ▶ 오류 없이 실행되면 설치 성공

NumPyro의 기본 사용법

구현된 알고리즘

- ▶ 해밀토니안 몬테 카를로 (HMC)
- ▶ 변분추론 (VI)

정규모형

▶ 모형

$$\begin{aligned}y_1, \dots, y_n &\sim \mathcal{N}(\mu, \sigma^2), \quad \sigma^2 = \frac{1}{\tau^2} \\ \mu &\sim \mathcal{N}(0, 10^2) \\ \tau^2 &\sim \mathcal{C}^+(0, 1)\end{aligned}$$

▶ 자료생성

```
import jax

n = 500
mu = 2
sigma = 1
y = mu + sigma*jax.random.normal(jax.random.PRNGKey(0), (n,))
```

모형정의

```
import numpyro as npr
import numpyro.distributions as dist

def normal_model(y):
    # 모델 파라미터들을 정의합니다.
    mu = npr.sample('mu', dist.Normal(0, 10))
    sigma = npr.sample('sigma', dist.HalfNormal(1))

    # 관측 데이터에 대한 확률 분포를 정의합니다.
    with npr.plate('data', len(y)):
        npr.sample('obs', dist.Normal(mu, sigma), obs=y)
```

해밀토니안 몬테 카를로 (HMC)

HMC: 사후표본 추출

```
from numpyro.infer import MCMC, NUTS

nuts_kernel = NUTS(normal_model)
mcmc = MCMC(nuts_kernel, num_samples=10000, num_warmup=5000,
            num_chains = 2)
mcmc.run(jax.random.PRNGKey(1), y=y)
```

HMC: 수렴성 판단 I

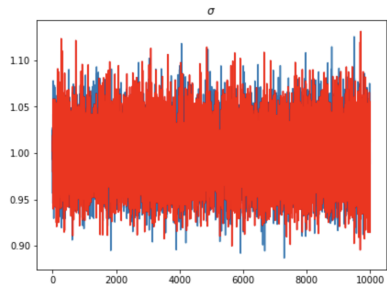
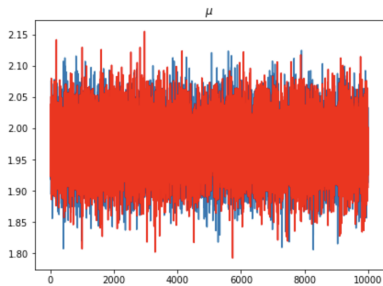
▶ 사후표본 추출

```
ps = mcmc.get_samples(group_by_chain=True)
```

▶ 자취그림

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize = (15,5))  
ax1.plot(ps['mu'][0]);ax1.plot(ps['mu'][1], 'r')  
ax1.set_title(r'$\mu$')  
ax2.plot(ps['sigma'][0]);ax2.plot(ps['sigma'][1], 'r')  
ax2.set_title(r'$\sigma$')  
plt.show()
```

HMC: 수렴성 판단 I



HMC: 수렴성 판단 II

- ▶ Gelman-Rubin 통계량

```
from numpyro.diagnostics import gelman_rubin

gelman_rubin(ps['mu'])
gelman_rubin(ps['sigma'])
```

- ▶ 유효 표본수

```
from numpyro.diagnostics import effective_sample_size

effective_sample_size(ps['mu'])
effective_sample_size(ps['sigma'])
```

HMC: 사후추론 I

▶ 요약통계량

```
mcmc.print_summary()
```

	mean	std	median	5.0%	95.0%	n_eff	r_hat
mu	1.97	0.05	1.97	1.90	2.04	18612.23	1.00
sigma	1.00	0.03	1.00	0.94	1.05	18188.41	1.00

Number of divergences: 0

HMC: 사후추론 II

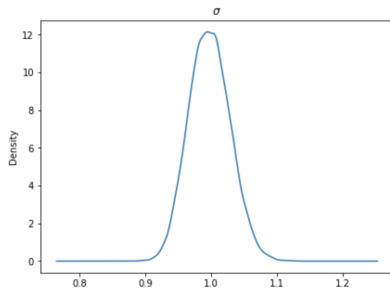
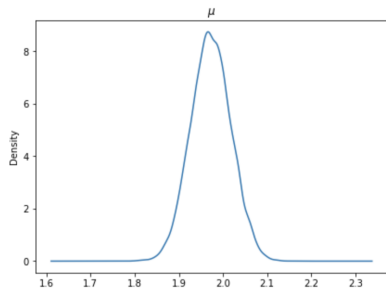
▶ 사후분포 그림

```
import pandas as pd

ps = mcmc.get_samples()
psd = pd.DataFrame(ps)

plt.figure(figsize = (15,5))
plt.subplot(1,2,1);psd['mu'].plot.kde()
plt.title(r'$\mu$')
plt.subplot(1,2,2);psd['sigma'].plot.kde()
plt.title(r'$\sigma$')
plt.show()
```

HMC: 사후추론 II



변분 추론 (VI)

VI: 변분분포 정의

```
from numpyro.infer.autoguide import AutoDiagonalNormal
from numpyro.infer.autoguide import AutoMultivariateNormal

guide = AutoDiagonalNormal(normal_model) # mean-field
```

VI: 모형 최적화 I

```
from numpyro.infer import SVI, Trace_ELBO
from numpyro.optim import Adam

# 최적화를 위한 Adam 옵티마이저 설정
optimizer = Adam(step_size=0.001)

# SVI 객체 생성
svi = SVI(normal_model, guide, optimizer, loss=Trace_ELBO())
```

VI: 모형 최적화 II

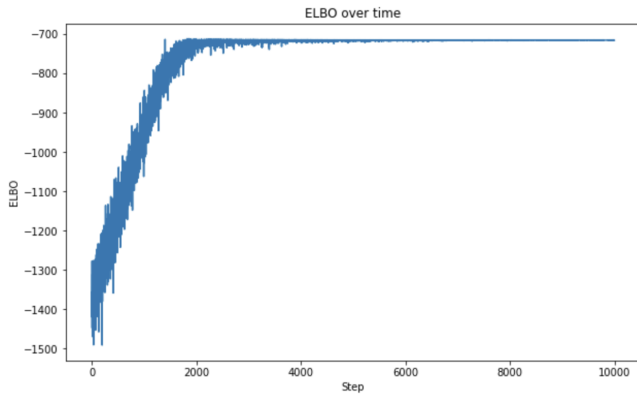
```
# SVI 실행
n_steps = 10000
svi_state = svi.init(jax.random.PRNGKey(0), y)

# 최적화 루프
elbo_values = []
for step in range(n_steps):
    svi_state, elbo = svi.update(svi_state, y)
    elbo_values.append(-elbo)
    if step % 1000 == 0:
        print('step {}: elbo = {}'.format(step, elbo))
```

VI: 수렴성 판단

```
plt.figure(figsize = (10,6))  
plt.plot(elbo_values)  
plt.title("ELBO over time")  
plt.xlabel("Step")  
plt.ylabel("ELBO")  
plt.show()
```

VI: 수렴성 판단



VI: 변분 파라미터

```
params = svi.get_params(svi_state)
print(params)
```

```
{'auto_loc': Array([ 1.9645712 , -0.00643641], dtype=float32), 'auto_scale': Array([0.04321681, 0.03311448], dtype=float32)}
```

추가 예제

베이지안 회귀모형

► Model:

$$y \sim \mathcal{N}\left(X\beta, \frac{1}{\tau}I_n\right),$$

where $y = (y_1, \dots, y_N)^\top$ and $X \in \mathbb{R}^{N \times p}$ is the design matrix.

► Priors:

$$\beta \sim \mathcal{N}\left(0, \frac{1}{\alpha}I_p\right), \quad \tau \sim \mathcal{Ga}(a_\tau, b_\tau).$$

► Variational distribution:

$$q_\lambda(\theta) = \mathcal{N}(\beta \mid \mu_{N,\beta}, \sigma_{N,\beta}^2) \mathcal{Ga}(\tau \mid a_{N,\tau}, b_{N,\tau}),$$

where $\theta = (\beta, \tau)^\top$ and $\lambda = (\mu_{N,\beta}, \sigma_{N,\beta}^2, a_{N,\tau}, b_{N,\tau})$.

베이지안 회귀모형 - 데이터 I

- ▶ WaffleDivorce dataset from Chapter 05, Statistical Rethinking (McElreath, R., 2016)
- ▶ The dataset contains divorce rates in each of the 50 states in the USA, along with predictors such as population, median age of marriage, whether it is a Southern state and, curiously, number of Waffle Houses.

베이지안 회귀모형 - 데이터 II

▶ <https://raw.githubusercontent.com/rmcelreath/rethinking/master/data/WaffleDivorce.csv>

```
import pandas as pd
```

```
DATASET_URL = "https://raw.githubusercontent.com/rmcelreath/rethinking/master/data/WaffleDivorce.csv"
```

```
dset = pd.read_csv(DATASET_URL, sep=";")
```

```
dset.head(10)
```

	Location	Loc	Population	MedianAgeMarriage	Marriage	Marriage SE	Divorce	Divorce SE	WaffleHouses	South	Slaves1860	Population1860	PropSlaves1860
0	Alabama	AL	4.78	25.3	20.2	1.27	12.7	0.79	128	1	435080	964201	0.450
1	Alaska	AK	0.71	25.2	26.0	2.93	12.5	2.05	0	0	0	0	0.000
2	Arizona	AZ	6.33	25.8	20.3	0.98	10.8	0.74	18	0	0	0	0.000
3	Arkansas	AR	2.92	24.3	26.4	1.70	13.5	1.22	41	1	111115	435450	0.260
4	California	CA	37.25	26.8	19.1	0.39	8.0	0.24	0	0	0	379994	0.000
5	Colorado	CO	5.03	25.7	23.5	1.24	11.6	0.94	11	0	0	34277	0.000
6	Connecticut	CT	3.57	27.6	17.1	1.06	6.7	0.77	0	0	0	460147	0.000
7	Delaware	DE	0.90	26.6	23.1	2.89	8.9	1.39	3	0	1798	112216	0.016
8	District of Columbia	DC	0.60	29.7	17.7	2.53	6.3	1.89	0	0	0	75080	0.000
9	Florida	FL	18.80	26.4	17.0	0.58	8.5	0.32	133	1	61745	140424	0.440

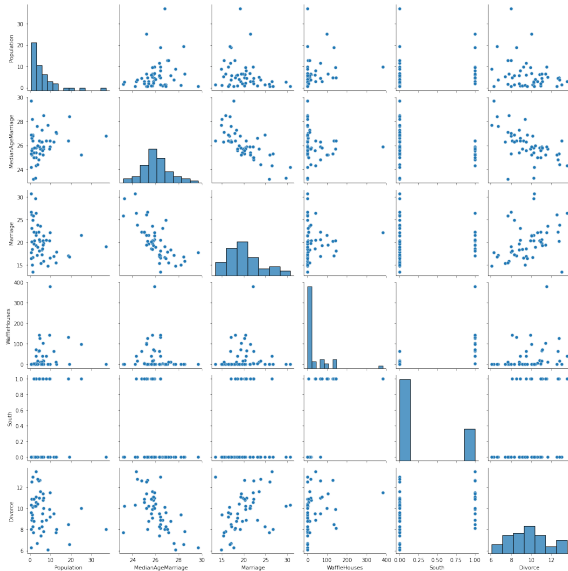
베이지안 회귀모형 - 데이터 III

```
import seaborn as sns

vars = [
    "Population",
    "MedianAgeMarriage",
    "Marriage",
    "WaffleHouses",
    "South",
    "Divorce",
]

sns.pairplot(dset, x_vars=vars, y_vars=vars);
```

베이지안 회귀모형 - 데이터 IV



베이지안 회귀모형 - 데이터 V

```
standardize = lambda x: (x - x.mean()) / x.std()

dset["AgeScaled"] = dset.MedianAgeMarriage.pipe(standardize)
dset["MarriageScaled"] = dset.Marriage.pipe(standardize)
dset["DivorceScaled"] = dset.Divorce.pipe(standardize)
```

베이지안 회귀모형 - 모형정의

```
import jax
import numpyro
import numpyro.distributions as dist

def regmodel(marriage=None, divorce=None):
    a = numpyro.sample("a", dist.Normal(0.0, 10))
    bM = numpyro.sample("bM", dist.Normal(0.0, 10))
    M = bM * marriage
    tausq = numpyro.sample("tausq", dist.Gamma(0.01, 0.01))
    sigma = jax.numpy.sqrt(1.0/tausq)
    mu = a + M
    with numpyro.plate('data', len(divorce)):
        numpyro.sample("obs", dist.Normal(mu, sigma), obs=divorce)
```

베이지안 회귀모형 - 변분분포

```
def guide(marriage=None, divorce=None):
    # tau
    alpha_tau = numpyro.param("alpha_tau", 1., constraint=dist.constraints.positive)
    beta_tau = numpyro.param("beta_tau", 1., constraint=dist.constraints.positive)
    numpyro.sample("tausq", dist.Gamma(alpha_tau, beta_tau))

    # a
    loc_a = numpyro.param('loc_a', 0.)
    scale_a = numpyro.param('scale_a', 1., constraint=dist.constraints.positive)
    numpyro.sample("a", dist.Normal(loc_a, scale_a))

    # bM
    loc_b = numpyro.param('loc_b', 0.)
    scale_b = numpyro.param('scale_b', 1., constraint=dist.constraints.positive)
    numpyro.sample("bM", dist.Normal(loc_b, scale_b))
```

베이지안 회귀모형 - 최적화

```
from numpyro.infer import SVI, Trace_ELBO
from numpyro.optim import Adam

# 최적화를 위한 Adam 옵티마이저 설정
optimizer = Adam(step_size=0.001)

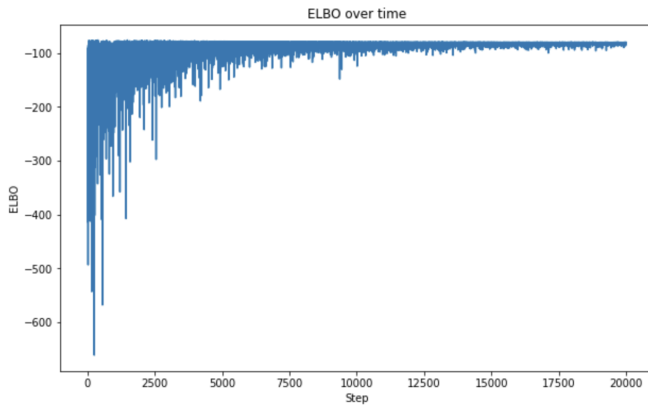
# SVI 객체 생성
svi = SVI(regmodel, guide, optimizer, loss=Trace_ELBO())

# SVI 실행
n_steps = 10000
svi_state = svi.init(jax.random.PRNGKey(0), dset.MarriageScaled.values,
                    divorce=dset.DivorceScaled.values)

# 최적화 루프
elbo_values = []
for step in range(n_steps):
    svi_state, elbo = svi.update(svi_state, dset.MarriageScaled.values,
                                divorce=dset.DivorceScaled.values)

    elbo_values.append(-elbo)
    if step % 1000 == 0:
        print('step {}: elbo = {}'.format(step, elbo))
```

베이지안 회귀모형 - 수렴성 판단



정리 및 다음 주

정리

- ▶ NumPyro (Pyro)는 Python을 위한 확률적 프로그래밍 라이브러리로 베이지안 모델링 및 추론에 사용된다.
- ▶ MCMC, 변분 추론 등 다양한 추론 방법을 제공한다.

다음 주

▶ Dirichlet process

참고문헌

- ▶ NumPyro tutorials
(<https://num.pyro.ai/en/stable/tutorials/>)
- ▶ Pyro tutorials (<https://pyro.ai/examples/>)